# Gestione chiamate a funzione

# Istruzioni necessarie

**Per chiamare la funzione/procedura**

`jal rd, etichetta`

ricorda nel registro **`rd`** la posizione dell'istruzione successiva

cambia il PC per iniziare l'esecuzione del corpo della funzione

( **rd <- PC+4** )

( **PC <- PC + etichetta** )

**Per tornare e continuare l'esecuzione del programma chiamante**

`jalr rd, imm(rs)` salta all'indirizzo contenuto nel registro indicato con un eventuale offset

( **rd <- PC + 4**)

( **PC <- rs + imm)**

**Come passare valori ALLA funzione (caso semplice)**

`a0`,…,`a7` 7 registri per passare fino a 7 valori a 32 bit

**e tornarli DALLA funzione**

`a0`, `a1` 2 registri per tornare fino a 2 valori a 32 bit

# Pseudoistruzioni

**Per chiamare la funzione/procedura**

`jal` ***`etichetta`***

ricorda nel registro **`ra`** la posizione dell'istruzione successiva
cambia il PC per iniziare l'esecuzione del corpo della funzione

( **ra <- PC+4** )
( **PC <- PC + etichetta** )

**Per tornare e continuare l'esecuzione del programma chiamante**

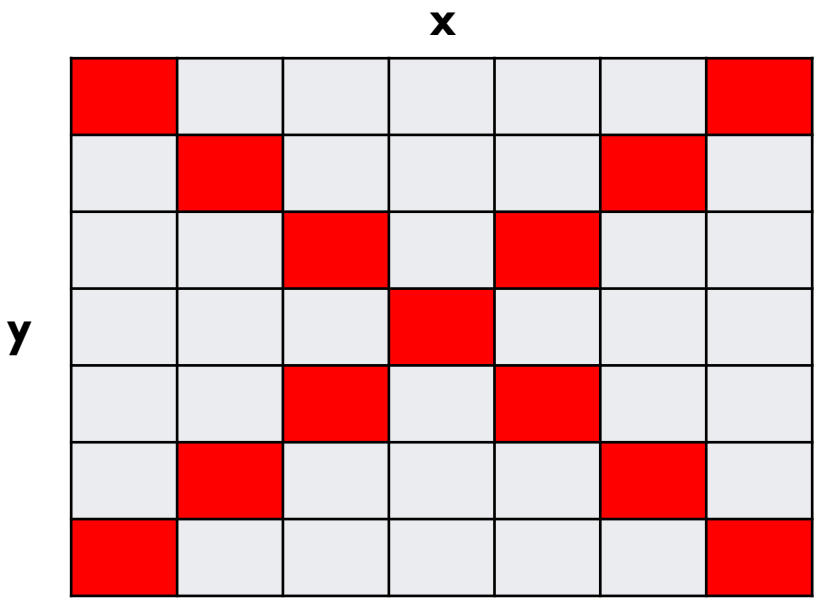
| | | |
|---|---|---|
| `jalr rs` | Salva in **`ra`** l'indirizzo dell'istruzione successiva e<br>salta all'indirizzo contenuto nel registro indicato | ( **ra <- PC+4** )<br>( **PC <- rs)** |
| `jr rs` | salta all'indirizzo contenuto nel registro indicato | ( **PC <- rs)** |
| `ret` | salta all'indirizzo contenuto nel registro **`ra`** | ( **PC <- ra)** |

# Esercizio

Calcolare e stampare la somma delle **due** diagonali di una matrice quadrata di word

Fate attenzione a non sommare due volte l'elemento centrale in caso di matrici di lato dispari

Suggerimento: scandite tutta la matrice e individuate le caselle sulle due diagonali



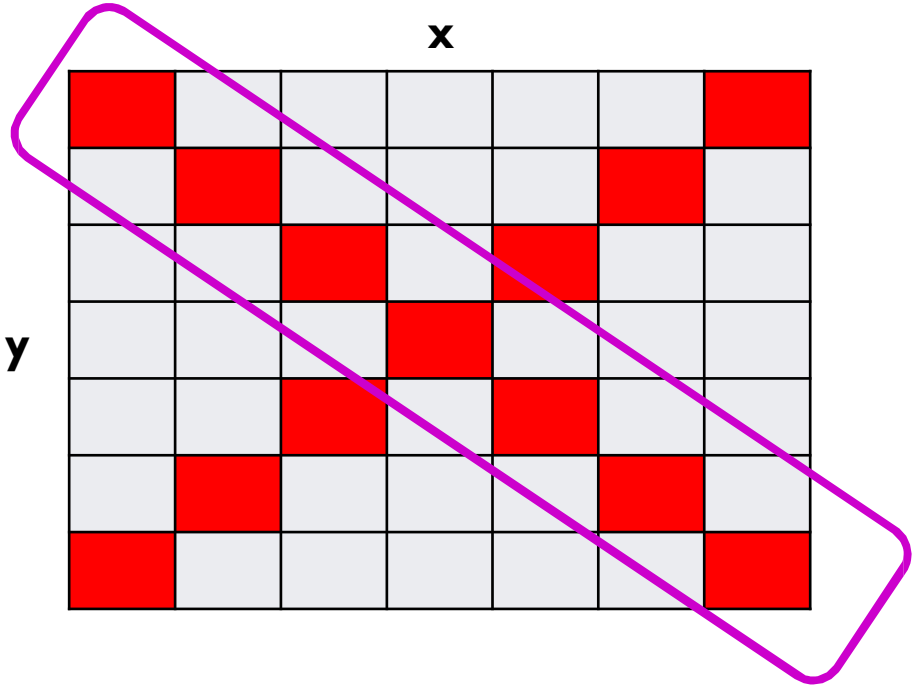
# Esercizio

Calcolare e stampare la somma delle **due** diagonali di una matrice quadrata di word

Fate attenzione a non sommare due volte l'elemento centrale in caso di matrici di lato dispari

Suggerimento: scandite tutta la matrice e individuate le caselle sulle due diagonali

**x = y**



# Esercizio

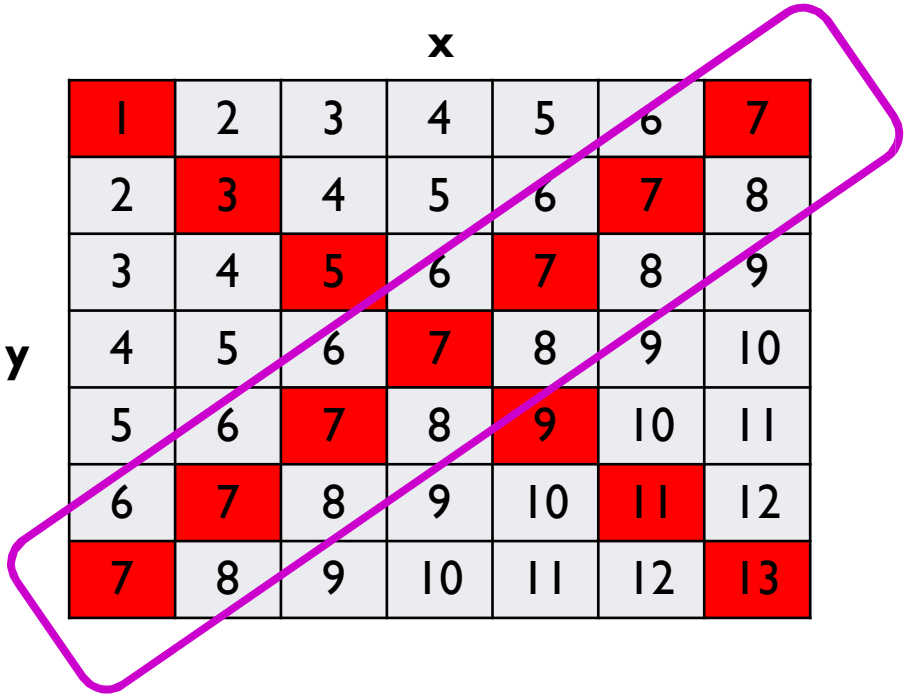
Calcolare e stampare la somma delle **due** diagonali di una matrice quadrata di word

Fate attenzione a non sommare due volte l'elemento centrale in caso di matrici di lato dispari

Suggerimento: scandite tutta la matrice e individuate le caselle sulle due diagonali

**x = y**

**x+y+1 = lato**



# Soluzione con funzioni

Definiamo la funzione **is_diagonal** che torna **1** (vero) se la casella a coordinate **x, y** si trova su una delle due diagonali principali altrimenti torna **0** (falso), con argomenti:

```
# a0    coordinata x
# a1    coordinata y
# a2    lato della matrice
```

# Soluzione con funzioni

Definiamo la funzione **is_diagonal** che torna **1** (vero) se la casella a coordinate **x, y** si trova su una delle due diagonali principali altrimenti torna **0** (falso), con argomenti:

```
        # a0    coordinata x
        # a1    coordinata y
        # a2    lato della matrice
```

```
is_diagonal:
        beq     a0, a1, yes     # se x=y siamo sulla prima diagonale
```

# Soluzione con funzioni

Definiamo la funzione **is_diagonal** che torna **1** (vero) se la casella a coordinate **x, y** si trova su una delle due diagonali principali altrimenti torna **0** (falso), con argomenti:

```
        # a0    coordinata x
        # a1    coordinata y
        # a2    lato della matrice
```

```
is_diagonal:
        beq     a0, a1, yes     # se x=y siamo sulla prima diagonale
```

```
yes:
        li      a0, 1           # il risultato è 1 (vero)
         # ritorno all'istruzione successiva alla chiamata
        jr      ra
```

# Soluzione con funzioni

Definiamo la funzione **is_diagonal** che torna **1** (vero) se la casella a coordinate **x, y** si trova su una delle due diagonali principali altrimenti torna **0** (falso), con argomenti:

```
        # a0    coordinata x
        # a1    coordinata y
        # a2    lato della matrice
is_diagonal:
        beq     a0,  a1,  yes  # se x=y siamo sulla prima diagonale
        add     a0,  a0,  a1   # altrimenti se la somma x+y+1
        addi    a0,  a0,  1
        beq     a0,  a2,  yes  # è uguale al lato siamo sulla seconda




yes:
        li      a0, 1          # il risultato è 1 (vero)
         # ritorno all'istruzione successiva alla chiamata
        jr      ra
```

# Soluzione con funzioni

Definiamo la funzione **is_diagonal** che torna **1** (vero) se la casella a coordinate **x, y** si trova su una delle due diagonali principali altrimenti torna **0** (falso), con argomenti:

```
        # a0    coordinata x
        # a1    coordinata y
        # a2    lato della matrice
is_diagonal:
        beq     a0,  a1,  yes  # se x=y siamo sulla prima diagonale
        add     a0,  a0,  a1   # altrimenti se la somma x+y+1
        addi    a0,  a0,  1
        beq     a0,  a2,  yes  # è uguale al lato siamo sulla seconda
        # altrimenti non siamo sulle due diagonali principali
        li a0, 0          # il risultato è 0
        ret               # ritorno all'istruzione successiva alla chiamata


yes:
        li      a0, 1           # il risultato è 1 (vero)
         # ritorno all'istruzione successiva alla chiamata
        jr      ra
```

# lettura dell'elemento corrente

Definiamo una seconda funzione che legge dalla matrice l'elemento a coordinate **x,y** (con gli stessi argomenti dell'altra)

```
        #       a0      coordinata x
        #       a1      coordinata y
        #       a2      lato della matrice
leggi_elemento:
```

# lettura dell'elemento corrente

Definiamo una seconda funzione che legge dalla matrice l'elemento a coordinate **x,y**
(con gli stessi argomenti dell'altra)

```
        #       a0      coordinata x
        #       a1      coordinata y
        #       a2      lato della matrice
leggi_elemento:
        mul     t0, a1, a2          # y*LATO
        add     t0, a0, t0          # x + y*LATO
        slli    t0, t0, 2           # offset = 4 * (x + y * LATO)
```

# lettura dell'elemento corrente

Definiamo una seconda funzione che legge dalla matrice l'elemento a coordinate **x,y**
(con gli stessi argomenti dell'altra)

```
        #       a0      coordinata x
        #       a1      coordinata y
        #       a2      lato della matrice
leggi_elemento:
        mul     a0, a1, a2          # y*LATO
        add     t0, a0, t0          # x + y*LATO
        slli    t0, t0, 2           # offset = 4 * (x + y * LATO)
        lw      t0, matrice(t0)     # leggo matrice[x][y]
```

# lettura dell'elemento corrente

Definiamo una seconda funzione che legge dalla matrice l'elemento a coordinate **x,y**
(con gli stessi argomenti dell'altra)

```
        #       a0      coordinata x
        #       a1      coordinata y
        #       a2      lato della matrice
leggi_elemento:
        mul     t0, a1, a2          # y*LATO
        add     t0, a0, t0          # x + y*LATO
        slli    t0, t0, 2           # offset = 4 * (x + y * LATO)
        lw      t0, matrice(t0)     # leggo matrice[x][y]
        jr      ra              # torno  l'esecuzione al chiamante
```

# lettura dell'elemento corrente

Definiamo una seconda funzione che legge dalla matrice l'elemento a coordinate **x,y**
(con gli stessi argomenti dell'altra)

```
        #       a0      coordinata x
        #       a1      coordinata y
        #       a2      lato della matrice
leggi_elemento:
        mul     t0, a1, a2          # y*LATO
        add     t0, a0, t0          # x + y*LATO
        slli    t0, t0, 2           # offset = 4 * (x + y * LATO)
        lw      t0, matrice(t0)     # leggo matrice[x][y]
        jr      ra              # torno  l'esecuzione al chiamante
```

**NOTA: cerco di non usare registri ulteriori in modo da lasciare più libertà nel programma chiamante**

**NOTA: vedremo in seguito come preservare i registri**

# Programma principale

```
.text
main:  li      t0,   0                 # x = 0
       li      a1,   0                 # y = 0
       lw      a2,   LATO(zero)        # lato della matrice
       li      t1,   0                 # somma = 0
```

# Programma principale

```
.text
main:   li      t0,   0                 # x = 0
        li      a1,   0                 # y = 0
        lw      a2,   LATO(zero)        # lato della matrice
        li      t1,   0                 # somma = 0
cicloY:         beq     a1,   a2,   fine        # se ultima riga
```

# Programma principale

```
.text
main:   li      t0,   0                 # x = 0
        li      a1,   0                 # y = 0
        lw      a2,   LATO(zero)        # lato della matrice
        li      t1,   0                 # somma = 0
cicloY:         beq     a1,   a2,   fine        # se ultima riga
cicloX:         mv      a0,   t0                # passaggio x come
                                                # primo parametro
                beq     a0,   a2,   nextY       # se ultima colonna
```

# Programma principale

```
.text
main:  li      t0,   0                 # x = 0
       li      a1,   0                 # y = 0
       lw      a2,   LATO(zero)        # lato della matrice
       li      t1,   0                 # somma = 0
cicloY:        beq     a1,   a2,   fine        # se ultima riga
cicloX:        mv      a0,   t0                # passaggio x come
                                               # primo parametro
               beq     a0,   a2,   nextY       # se ultima colonna
         jal     is_diagonal             # test se sulla diagonale
         beqz    a0, nextX               # se falso prossima X
```

# Programma principale

```
.text
main:  li     t0,  0                 # x = 0
       li     a1,  0                 # y = 0
       lw     a2,  LATO(zero)        # lato della matrice
       li     t1,  0                 # somma = 0
cicloY:       beq    a1,  a2,  fine       # se ultima riga
cicloX:       mv     a0,  t0              # passaggio x come
                                          # primo parametro
              beq    a0,  a2,  nextY      # se ultima colonna
       jal    is_diagonal           # test se sulla diagonale
       beqz   a0, nextX             # se falso prossima X
       jal    leggi_elemento        # altrimenti leggi
       add    t1, t1, a0            # e somma l'elemento
```

# Programma principale

```
.text
main:  li     s0,  0                 # x = 0
       li     a1,  0                 # y = 0
       lw     a2,  LATO(zero)        # lato della matrice
       li     s1,  0                 # somma = 0
cicloY:       beq    a1,  a2,  fine       # se ultima riga
cicloX:       mv     a0,  s0              # passaggio x come
                                          # primo parametro
              beq    a0,  a2,  nextY      # se ultima colonna
       jal    is_diagonal           # test se sulla diagonale
       beqz   a0, nextX             # se falso prossima X
       jal    leggi_elemento        # altrimenti leggi
       add    s1, s1, a0            # e somma l'elemento
nextX: addi   s0, s0, 1             # x += 1 (prossima colonna)
       j      cicloX
```

# (segue)

```
nextY: addi   a1, a1,   1   # y += 1 (prossima riga)
       li     s0, 0         # x = 0  (colonna 0)
       j      cicloY
fine:
       li     a0,  10       # ecall 10 = fine
       ecall
```

```
.data
LATO:   .word  20                  # lato della matrice
matrice:       .skip  4*20*20      # matrice 20x20
```

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime

- ne vengono salvate altre

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime

- ne vengono salvate altre

- …

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime

- ne vengono salvate altre

- …

- vengono prima ripristinate le seconde

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
  - ne vengono salvate altre
    - …
  - vengono prima ripristinate le seconde
- e poi vengono ripristinate le prime

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
  - vengono salvate le seconde
    - vengono salvate le terze
      - …
    - vengono ripristinate le terze
  - vengono ripristinate le seconde
- e infine vengono ripristinate le prime

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante

-nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
  - vengono salvate le seconde
    - vengono salvate le terze
      - …
    - vengono ripristinate le terze
  - vengono ripristinate le seconde
- e infine vengono ripristinate le prime

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro `sp` (Stack Pointer)

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante
  - nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
  - ne vengono salvate altre
    - …
  - vengono prima ripristinate le seconde
- e poi vengono ripristinate le prime

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro `sp` (Stack Pointer)

| Address | MEM |
|---|---|
| 1000 | xxxx |
| 996 | yyyy |
| 992 | zzzz |
| 988 | tttt |
| 984 | ssss |
| 980 | rrrr |
| 976 | |
| 972 | |
| 968 | |
| 964 | |
| 960 | |
| 956 | |
| 952 | |
| 948 | |
| 944 | |
| 940 | |
| 936 | |
| 932 | |
| 928 | |
| 924 | |

# Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante
  - nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
- `sp` **980**
- …
- vengono ripristinate le seconde
- e infine vengono ripristinate le prime

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro `sp` (Stack Pointer)

| Address | MEM |
|---|---|
| 1000 | xxxx |
| 996 | yyyy |
| 992 | zzzz |
| 988 | tttt |
| 984 | ssss |
| 980 | rrrr |
| 976 | |
| 972 | |
| 968 | |
| 964 | |
| 960 | |
| 956 | |
| 952 | |
| 948 | |
| 944 | |
| 940 | |
| 936 | |
| 932 | |
| 928 | |
| 924 | |

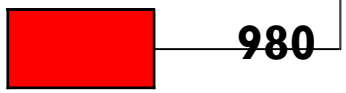
# Preservare il contenuto dei registri

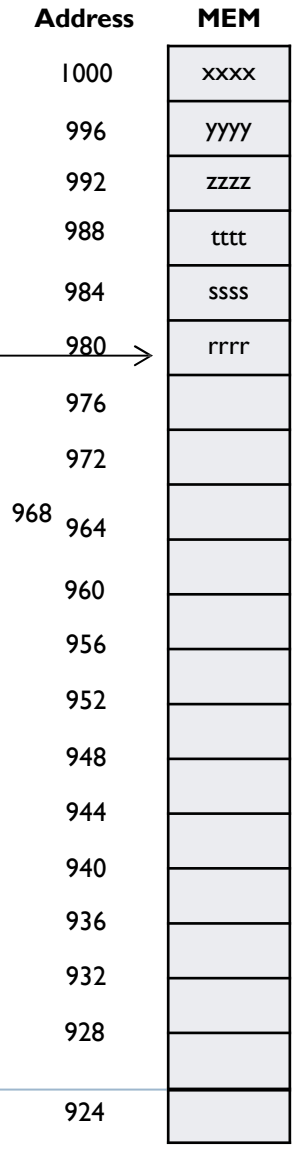
Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante
- nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di `ra`

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
- …
- vengono ripristinate le seconde
- e infine vengono ripristinate le prime



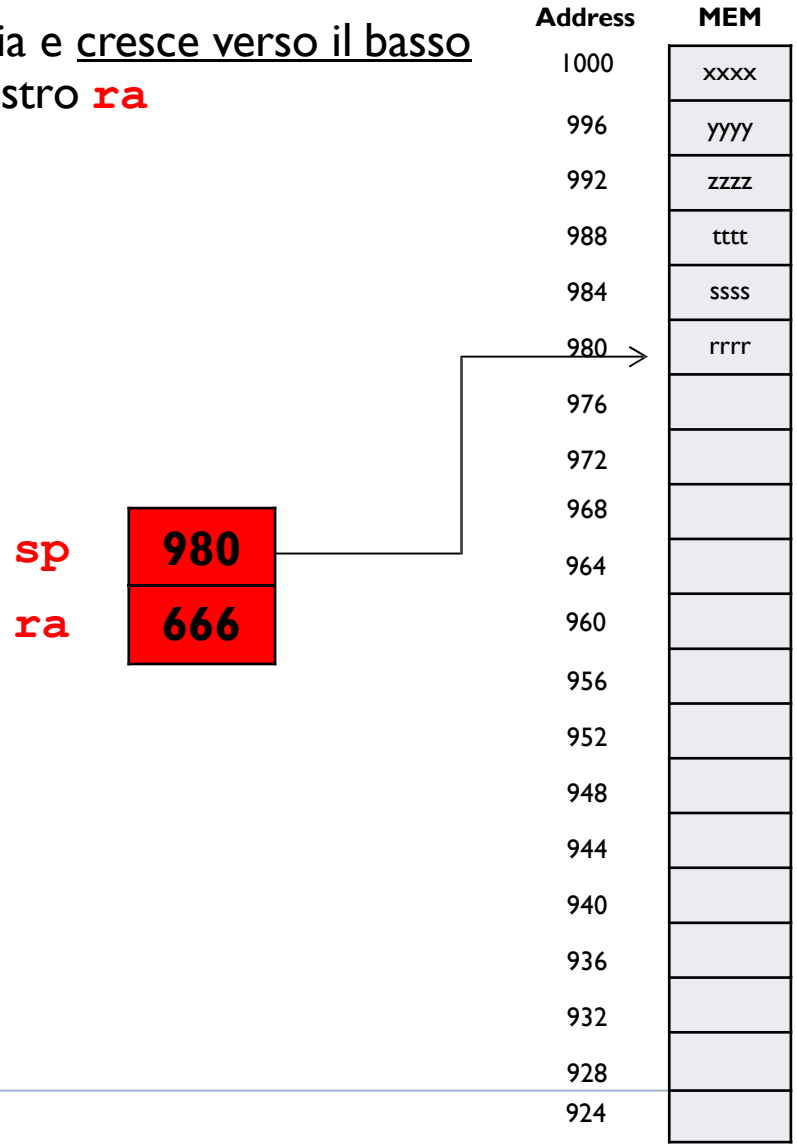




Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro `sp` (Stack Pointer)

# Uso dello stack

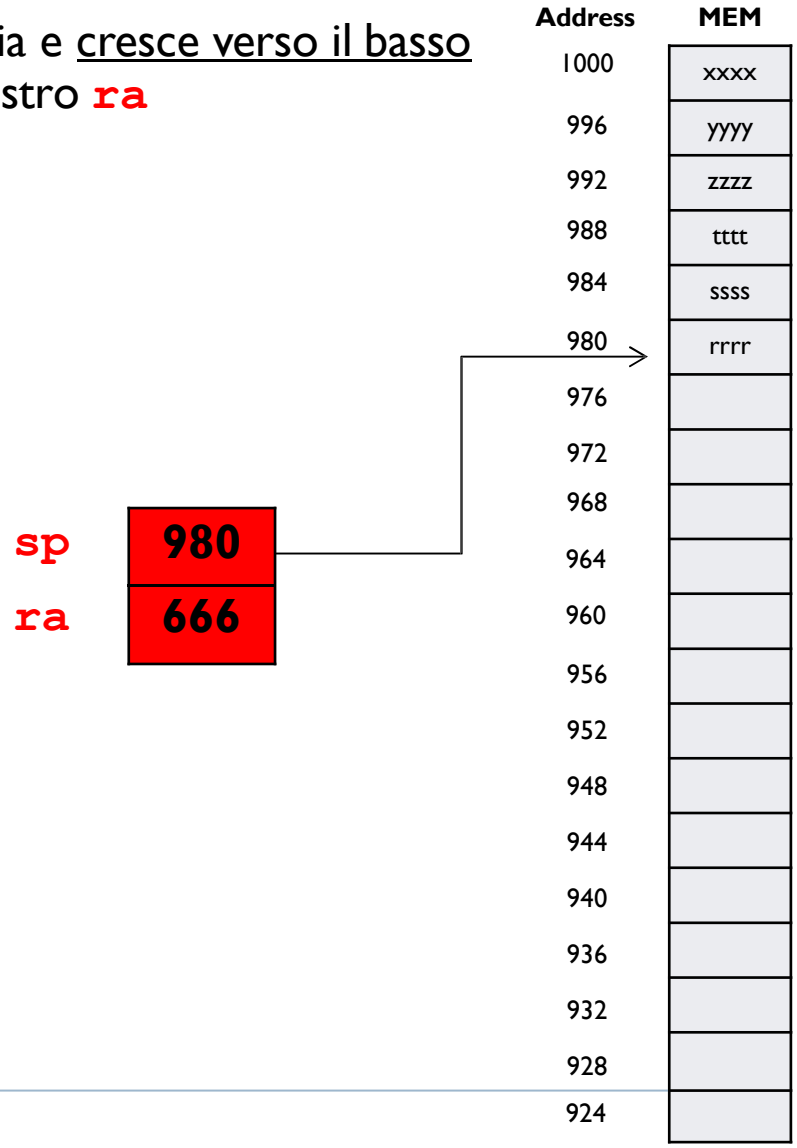
Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro **ra**



# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**



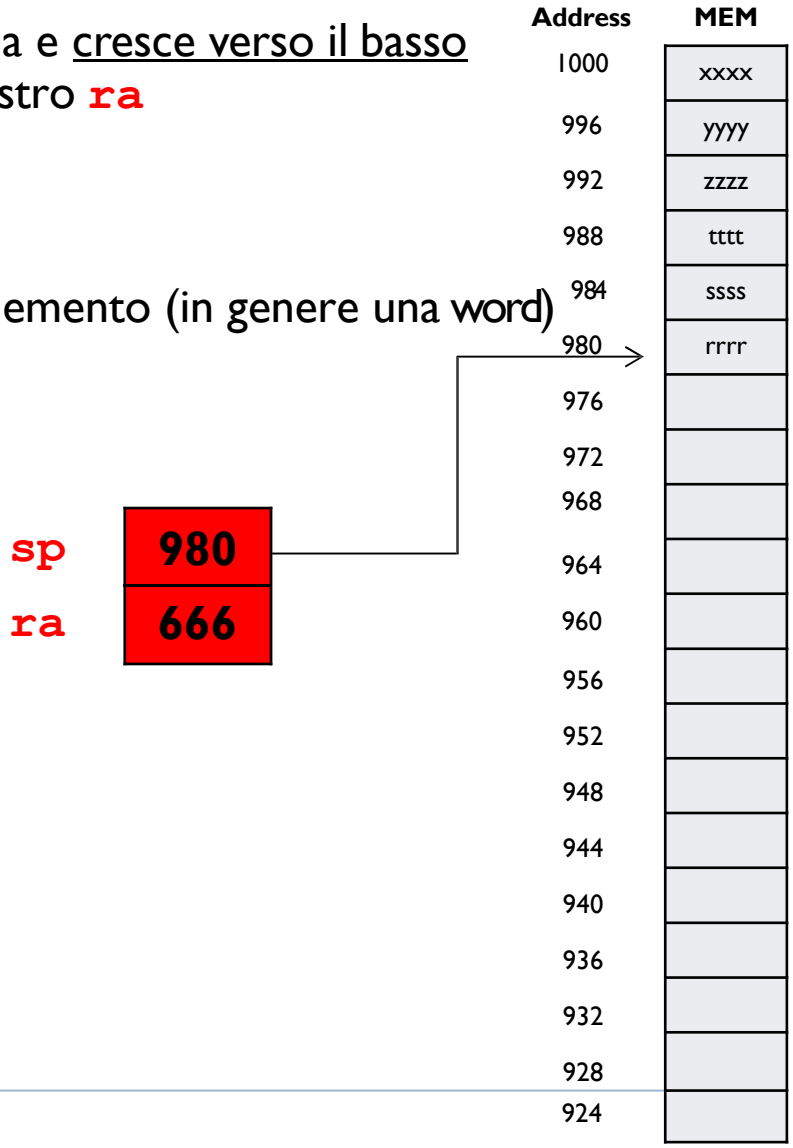
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)

```
addi    sp, sp, -4
```



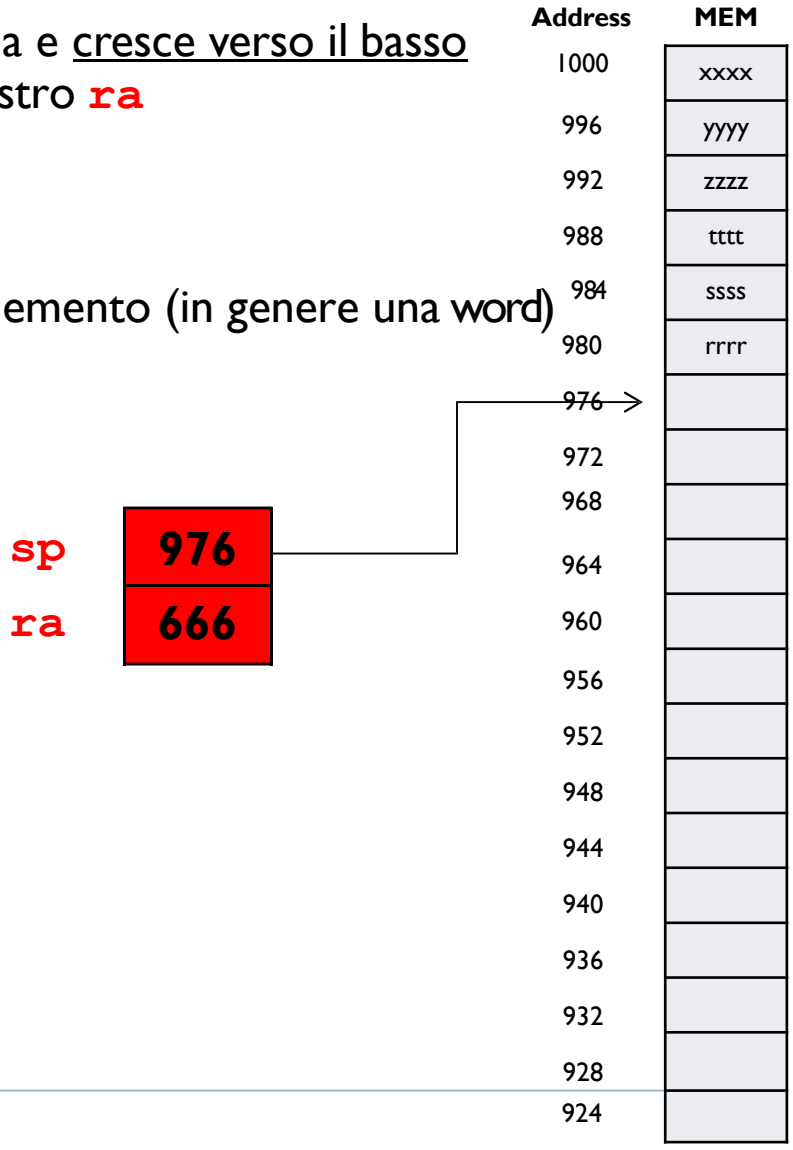
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)

```
addi   sp, sp, -4
```



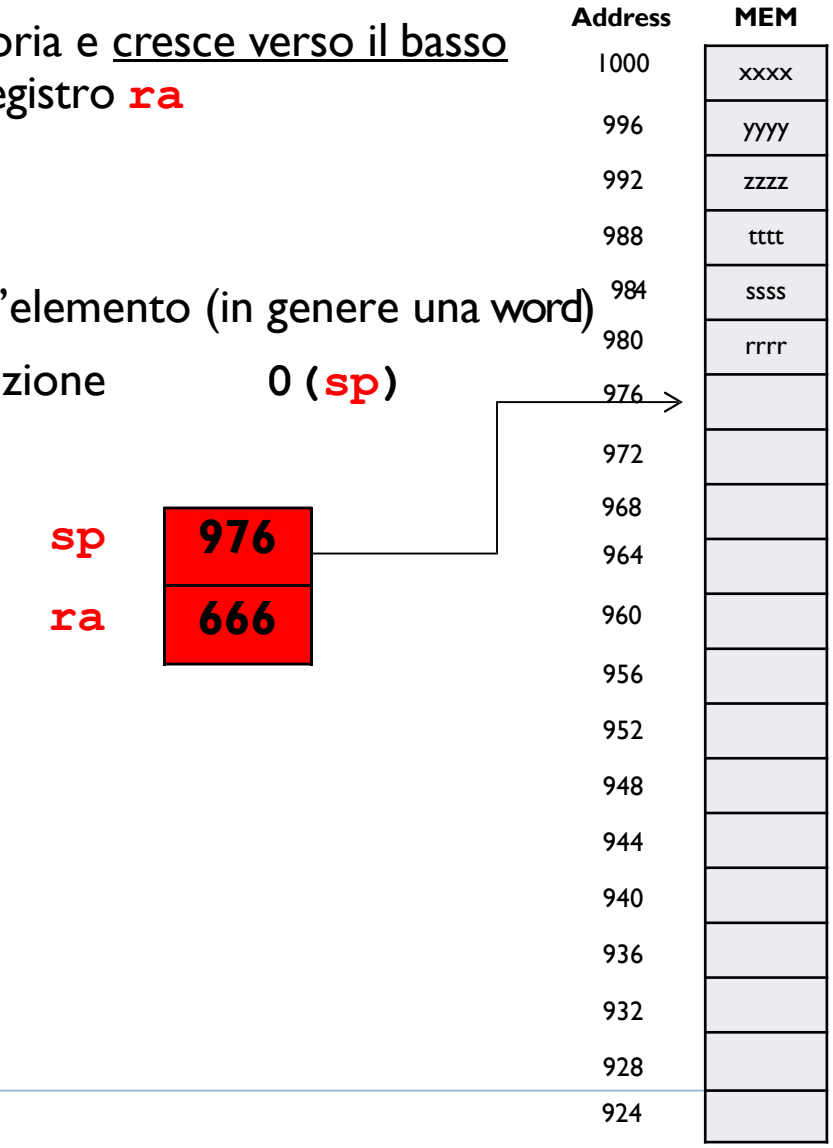
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione `0(sp)`

```
addi    sp, sp, -4
sw      ra, 0(sp)
```



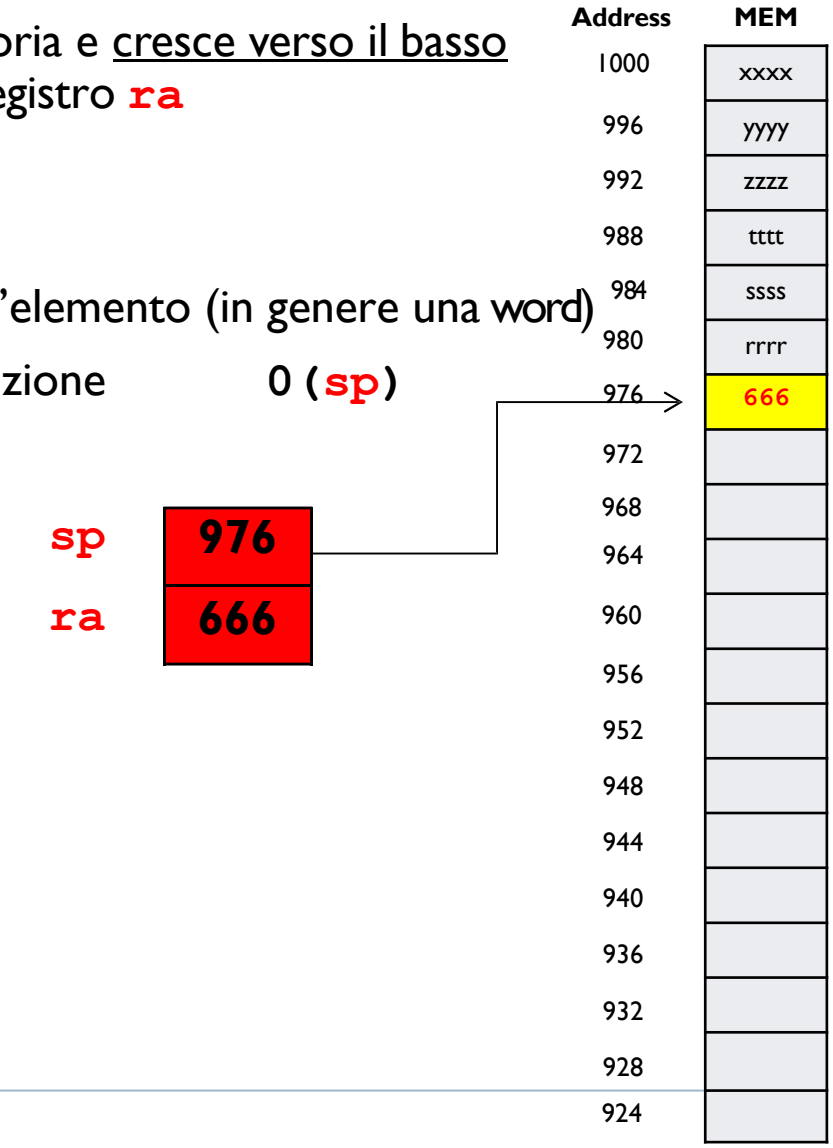
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione `0(sp)`

```
addi    sp, sp, -4
sw      ra, 0(sp)
```



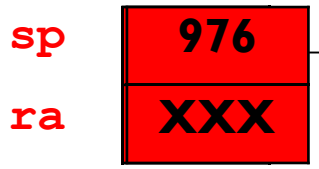
# Uso dello stack

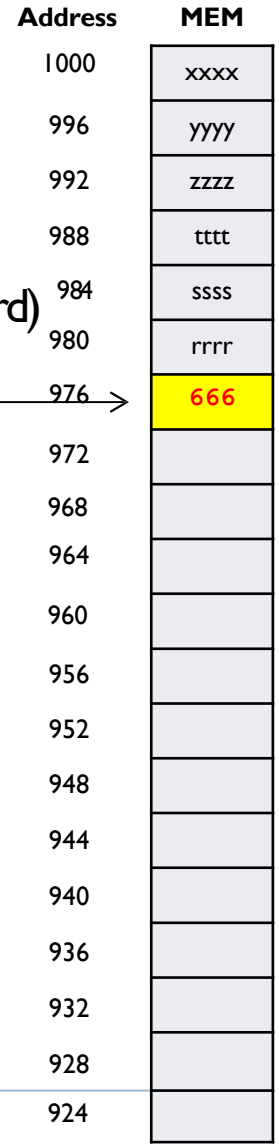
Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione `0(sp)`

```
addi    sp, sp, -4
sw      ra, 0(sp)
```




**Come recuperare un elemento (pop):**

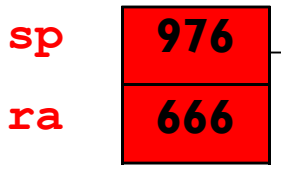
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione `0(sp)`

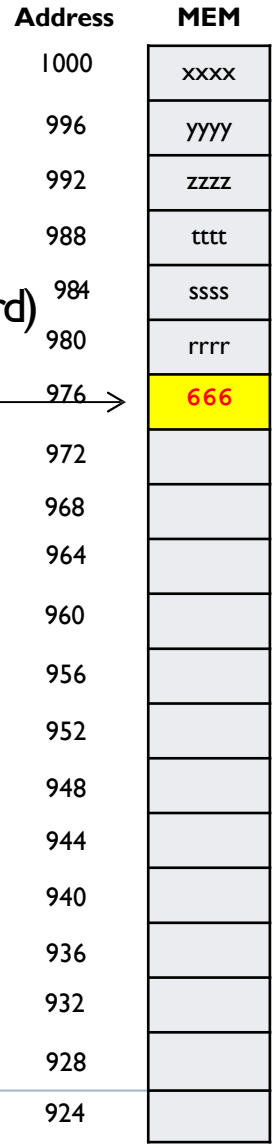
```
addi    sp, sp, -4
sw      ra, 0(sp)
```




**Come recuperare un elemento (pop):**

- si legge l'elemento dalla posizione `0(sp)`

```
lw      ra, 0(sp)
```



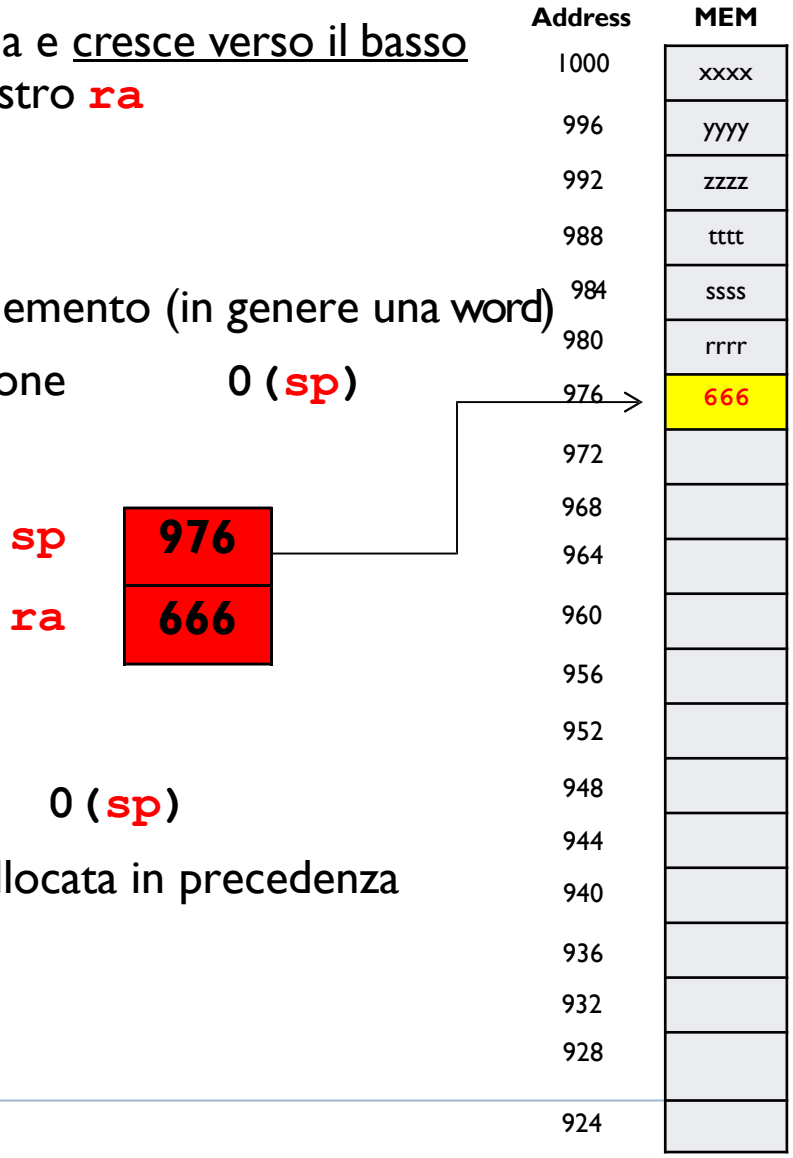
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione `0(sp)`

```
addi    sp, sp, -4
sw      ra, 0(sp)
```




**Come recuperare un elemento (pop):**

- si legge l'elemento dalla posizione `0(sp)`
- si incrementa lo `sp` della quantità allocata in precedenza

```
lw      ra, 0(sp)
addi    sp, sp, 4
```

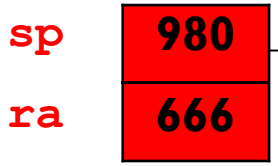
# Uso dello stack

Lo stack si trova nella parte «alta» della memoria e cresce verso il basso
Supponiamo di voler salvare e ripristinare il registro `ra`

**Come salvare un elemento (push):**

- si decrementa lo `sp` della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione `0(sp)`

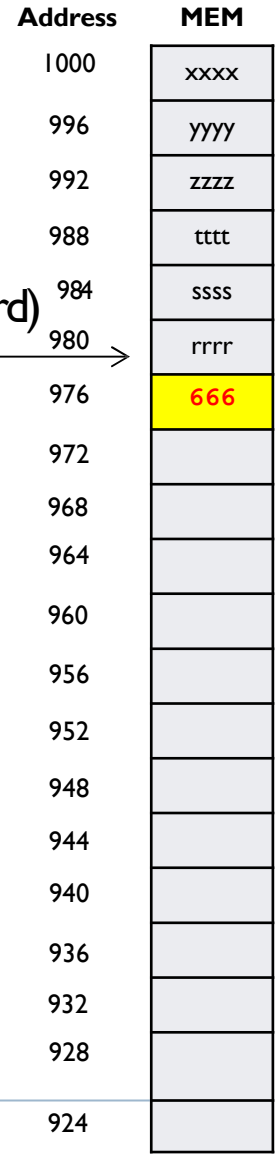
```
addi    sp, sp, -4
sw      ra, 0(sp)
```




**Come recuperare un elemento (pop):**

- si legge l'elemento dalla posizione `0(sp)`
- si incrementa lo `sp` della quantità allocata in precedenza

```
lw      ra, 0(sp)
addi    sp, sp, 4
```



# Uso dello stack in una funzione

**All'inizio della funzione:**

- **allocare** su stack abbastanza word da contenere i registri da preservare

```
funzione:
        addi    sp, sp, -12

# corpo della funzione
```

# Uso dello stack in una funzione

**All'inizio della funzione:**

-**allocare** su stack abbastanza word da contenere i registri da preservare

-**salvare** su stack i registri, ad offset multipli di 4 rispetto a `sp`

```
funzione:
        addi    sp, sp, -12
        sw      ra, 0(sp)
        sw      a0, 4(sp)
        sw      a1, 8(sp)


        # corpo della funzione
```

# Uso dello stack in una funzione

**All'inizio della funzione:**

-**allocare** su stack abbastanza word da contenere i registri da preservare

-**salvare** su stack i registri, ad offset multipli di 4 rispetto a `sp`

**NOTA:** conviene allocare tutto lo spazio assieme per avere **offset che restano costanti** durante tutta l'esecuzione della funzione

```
funzione:
        addi    sp, sp, -12
        sw      ra, 0(sp)
        sw      a0, 4(sp)
        sw      a1, 8(sp)


        # corpo della funzione
```

# Uso dello stack in una funzione

**All'inizio della funzione:**

-**allocare** su stack abbastanza word da contenere i registri da preservare

-**salvare** su stack i registri, ad offset multipli di 4 rispetto a `sp`

**NOTA:** conviene allocare tutto lo spazio assieme per avere **offset che restano costanti** durante tutta l'esecuzione della funzione

**All'uscita della funzione:**

- **ripristinare** da stack i registri salvati, agli stessi offset usati precedentemente

```
funzione:
        addi    sp, sp, -12
        sw      ra, 0(sp)
        sw      a0, 4(sp)
        sw      a1, 8(sp)

        # corpo della funzione

        lw      ra, 0(sp)
        lw      a0, 4(sp)
        lw      a1, 8(sp)
```

# Uso dello stack in una funzione

**All'inizio della funzione:**

-**allocare** su stack abbastanza word da contenere i registri da preservare

-**salvare** su stack i registri, ad offset multipli di 4 rispetto a `sp`

**NOTA:** conviene allocare tutto lo spazio assieme per avere **offset che restano costanti** durante tutta l'esecuzione della funzione

**All'uscita della funzione:**

-**ripristinare** da stack i registri salvati, agli stessi offset usati precedentemente

-**disallocare** da stack lo stesso spazio allocato in precedenza

```
funzione:
        addi    sp, sp, -12
        sw      ra, 0(sp)
        sw      a0, 4(sp)
        sw      a1, 8(sp)

        # corpo della funzione

        lw      ra, 0(sp)
        lw      a0, 4(sp)
        lw      a1, 8(sp)
        addi    sp, sp, 12
```

# Uso dello stack in una funzione

**All'inizio della funzione:**

-**allocare** su stack abbastanza word da contenere i registri da preservare

-**salvare** su stack i registri, ad offset multipli di 4 rispetto a `sp`

**NOTA:** conviene allocare tutto lo spazio assieme per avere **offset che restano costanti** durante tutta l'esecuzione della funzione

**All'uscita della funzione:**

-**ripristinare** da stack i registri salvati, agli stessi offset usati precedentemente

-**disallocare** da stack lo stesso spazio allocato in precedenza

- **tornare** alla funzione chiamante

```
funzione:
        addi    sp, sp, -12
        sw      ra, 0(sp)
        sw      a0, 4(sp)
        sw      a1, 8(sp)

        # corpo della funzione

        lw      ra,  0(sp)
        lw      a0,  4(sp)
        lw      a1,  8(sp)
        addi    sp,  sp, 12
        jr      ra              # oppure ret
```

## Record di attivazione

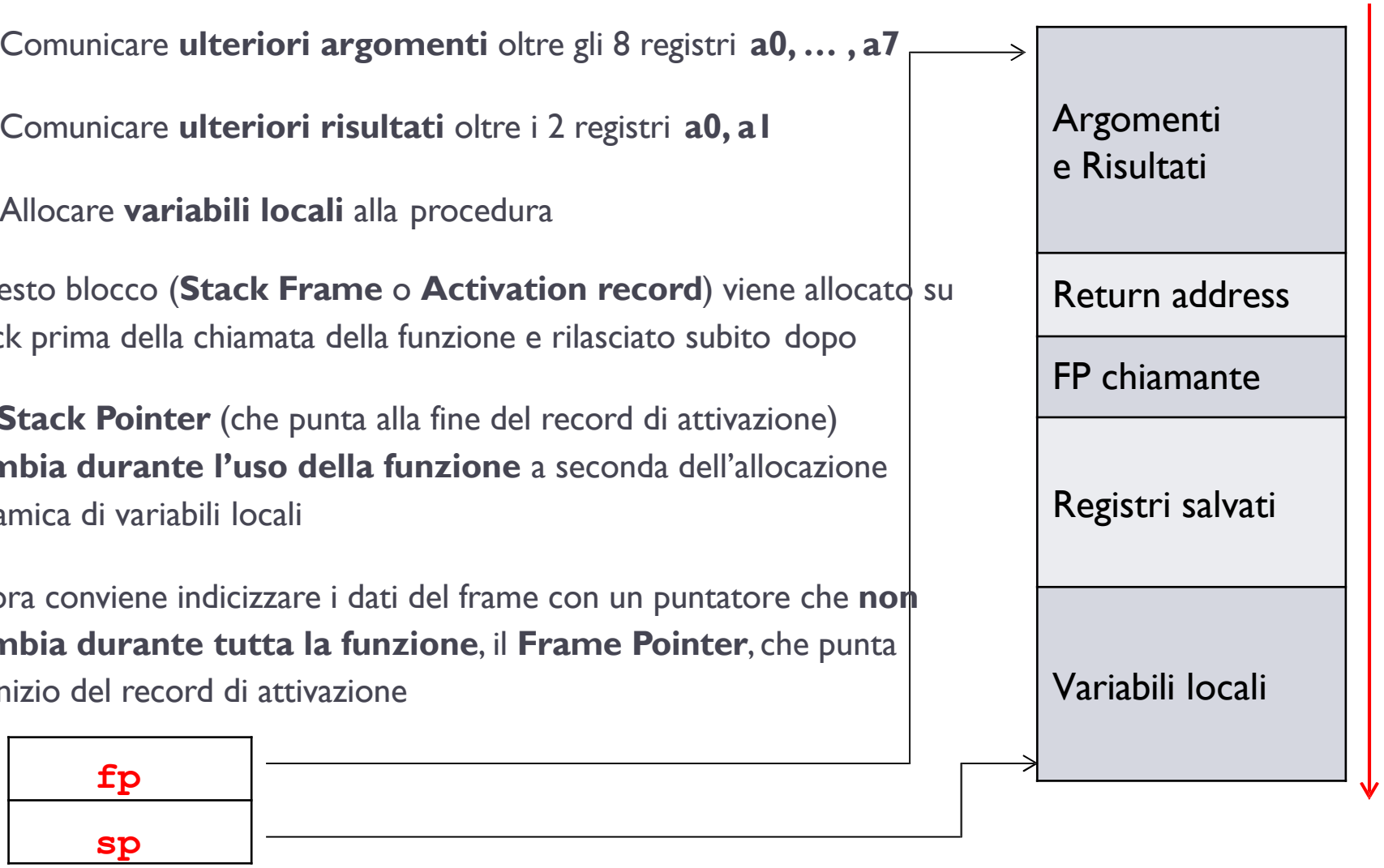
La stack è usata anche per:

- Comunicare **ulteriori argomenti** oltre gli 8 registri **a0, … , a7**
- Comunicare **ulteriori risultati** oltre i 2 registri **a0, a1**
- Allocare **variabili locali** alla procedura

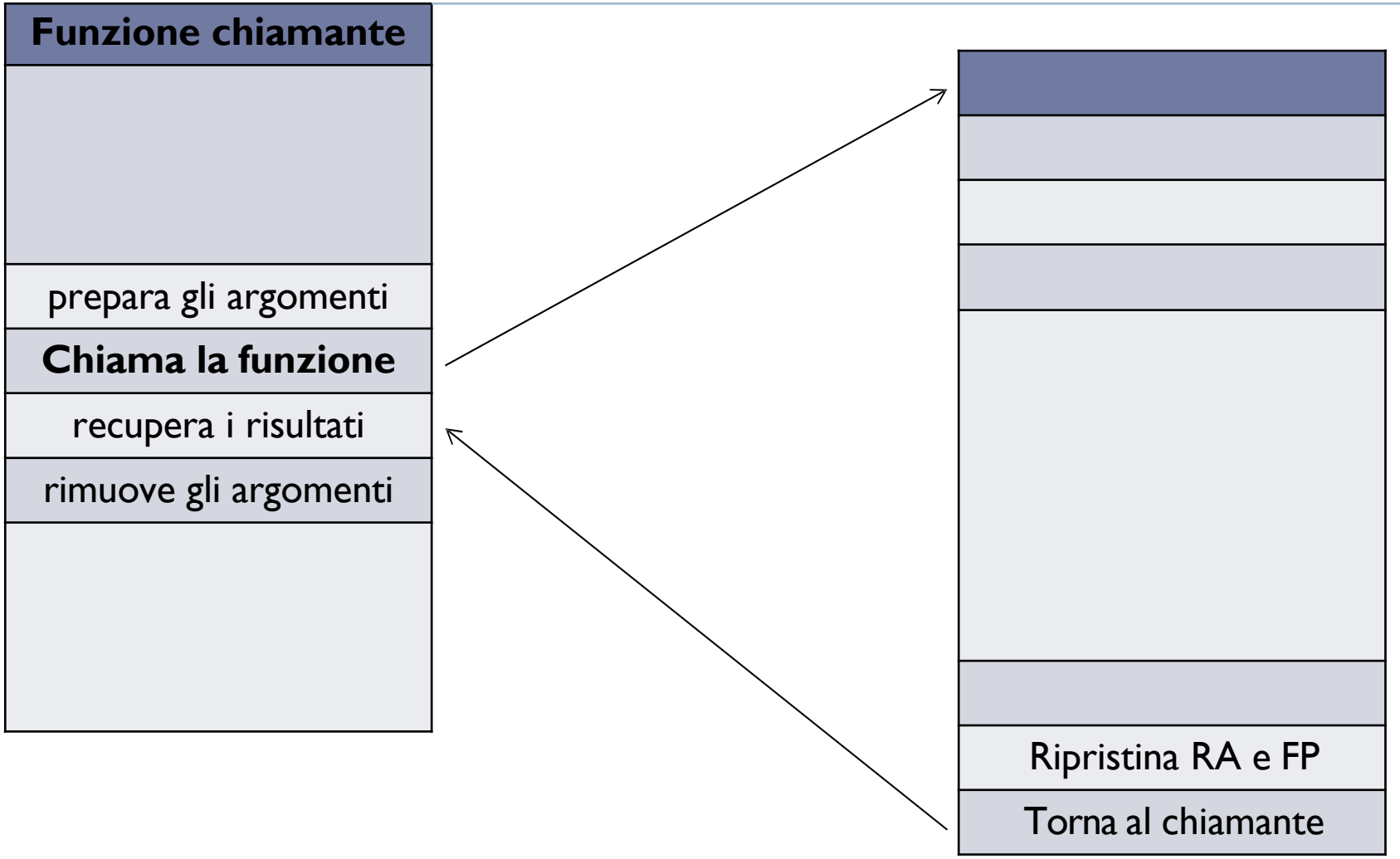
Questo blocco (**Stack Frame** o **Activation record**) viene allocato su stack prima della chiamata della funzione e rilasciato subito dopo

Lo **Stack Pointer** (che punta alla fine del record di attivazione) **cambia durante l'uso della funzione** a seconda dell'allocazione dinamica di variabili locali

Allora conviene indicizzare i dati del frame con un puntatore che **non cambia durante tutta la funzione**, il **Frame Pointer**, che punta all'inizio del record di attivazione



# Schema di una chiamata



# Linker

- Programma che risolve le etichette ancora presenti nelle istruzioni e genera un file eseguibile
  - Risolvere significa assegnare un indirizzo di memoria a quelle etichette
- I riferimenti non risolti sono contenuti nella tabella dei simboli
- Due tipi
  - Statico
    - Risoluzione di tutte le etichette prima che il programma venga eseguito
  - Dinamico
    - Risoluzione a runtime delle etichette in uso con approccio «lazy»
    - Introduzione di una procedura *dummy* che mappa la funzione desiderata F in memoria e poi la chiama
      - Nelle successive chiamate a F non c'è più bisogno di chiamare quella dummy, ma si chiama direttamente F

# Linker

- Esempio: la procedura A contiene un riferimento alla procedura B

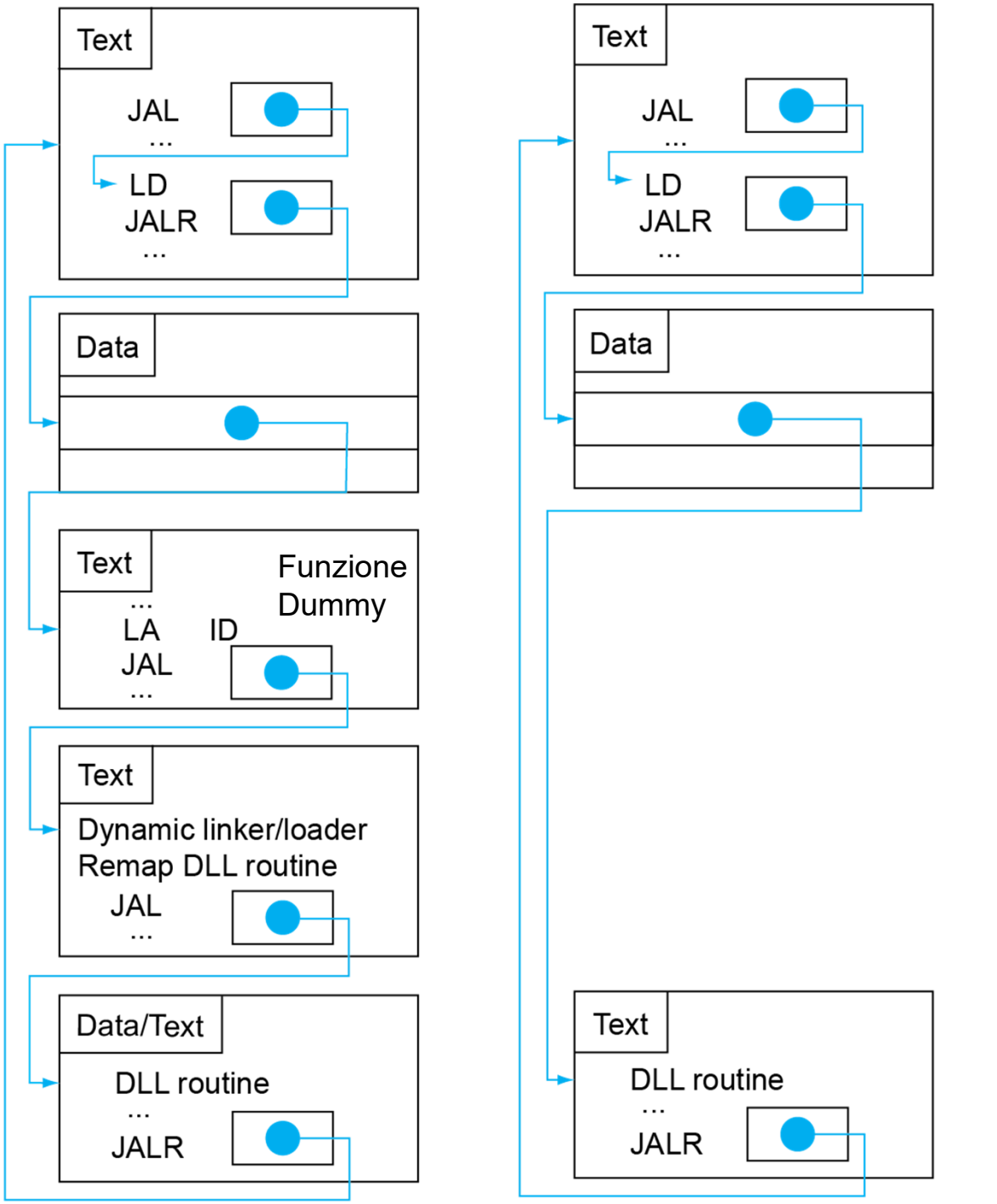
```
A:
  lw s0, 0(t1)
  jal ra, B
```

- NOTA: nell'esempio viene usato il linguaggio assembler, ma nella realtà le istruzioni sono sequenze di bit.
- Nel file eseguibile che il linker va a generare ci sarà la procedura A e la procedura B all'interno del segmento di testo.
  - Supponiamo che
    - la procedura A sia lunga 0x100,
    - abbia la prima istruzione a 0x400000, quindi l'istruzione jal è all'indirizzo 0x400004,
    - la procedura B sia la successiva.
  - La procedura B inizierà a 0x400100
  - Il linker aggiorna l'istruzione jal della procedura A in questo modo:

    ```
    jal ra, 0xfc
    ```

    - Dove 0xfc è la differenza tra l'indirizzo della prima istruzione di B e l'istruzione jal di A, quindi (0x400100 – 0x400004)
- Analoghi ragionamenti per risolvere i riferimenti sui dati.

# Caricamento Dinamico




(a) First call to DLL routine

(b) Subsequent calls to DLL routine